Hadoop Distributed Systems — Lab Final Study Guide

Your installation PDF covers *setup*. This guide covers what you'll actually be asked to **do** in a lab exam: HDFS operations, running MapReduce jobs, writing your own MapReduce program, YARN monitoring, web UIs, and the troubleshooting moves examiners love to test. Assume your cluster is already installed and configured as in your PDF.

---

1. Pre-Exam Startup Checklist

Run this the moment you sit down — most lab marks are lost to "my cluster won't start."

```
sudo service ssh start
start-dfs.sh
start-yarn.sh
jps
```

Expected `jps` output (order doesn't matter):

```
NameNode
DataNode
SecondaryNameNode
ResourceManager
NodeManager
Jps
```

If any daemon is missing, see **Section 7 — Troubleshooting** before doing anything else.

---

2. HDFS Command Reference (Full Set)

You already have the basics (`-mkdir`, `-ls`, `-put`, `-get`, `-cat`, `-cp`, `-mv`, `-rm`). Here's everything else examiners like to test:

Command	Purpose
`hdfs dfs -mkdir -p /a/b/c`	Create nested directories in one shot
`hdfs dfs -copyFromLocal file.txt /input`	Same as `-put`
`hdfs dfs -copyToLocal /input/file.txt .`	Same as `-get`
`hdfs dfs -moveFromLocal file.txt /input`	Upload, then **delete** local copy
`hdfs dfs -appendToFile local.txt /input/file.txt`	Append local content to an HDFS file
`hdfs dfs -tail /input/file.txt`	Show last KB of a file
`hdfs dfs -touchz /input/empty.txt`	Create a zero-byte file
`hdfs dfs -getmerge /input/* merged.txt`	Merge multiple HDFS files into one local file
`hdfs dfs -count /input`	Count dirs, files, and total bytes under a path
`hdfs dfs -stat /input/file.txt`	Show file metadata (size, mod time)

`hdfs dfs -chmod 755 /input/file.txt`	Change permissions
`hdfs dfs -chown user:group /input/file.txt`	Change owner/group
`hdfs dfs -setrep -w 2 /input/file.txt`	Change replication factor for one file
`hdfs dfs -expunge`	Empty the HDFS trash
`hdfs dfs -du -h /`	Human-readable disk usage per path
`hdfs dfs -df -h /`	Overall filesystem capacity
`hdfs dfs -rm -skipTrash /input/file.txt`	Permanently delete, bypassing trash
`hdfs dfsadmin -report`	Full cluster health report (capacity, live/dead nodes)
`hdfs dfsadmin -safemode get`	Check if NameNode is in safe mode
`hdfs dfsadmin -safemode leave`	Force NameNode out of safe mode
`hdfs fsck /input -files -blocks -locations`	Filesystem check — verify block health & locations
`hdfs balancer`	Rebalance blocks across DataNodes (no-op on single-node)

Quick round-trip drill you should be able to do blindfolded:

```
        hdfs dfs -mkdir -p /exam/input
        echo "hadoop is a distributed system" > local.txt
        hdfs dfs -put local.txt /exam/input
        hdfs dfs -ls /exam/input
        hdfs dfs -cat /exam/input/local.txt
        hdfs dfs -get /exam/input/local.txt ./downloaded.txt
```

---

3. Running MapReduce Jobs — Built-in Examples

Hadoop ships a jar full of ready-made jobs. This is the fastest way to *prove YARN works* in an exam.

```
        # Locate the examples jar (version number matches your Hadoop install)
        find $HADOOP_HOME/share/hadoop/mapreduce -name "*examples*.jar"
```

WordCount (the classic)

```
        hdfs dfs -mkdir -p /wc/input
        hdfs dfs -put local.txt /wc/input
        hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar \
          wordcount /wc/input /wc/output

        hdfs dfs -ls /wc/output
        hdfs dfs -cat /wc/output/part-r-00000
```

Note: the output directory (`/wc/output`) must **not already exist** — MapReduce refuses to overwrite it. Delete it first if re-running: `hdfs dfs -rm -r /wc/output`.

Pi estimation (tests YARN/compute, no HDFS file needed)

```
        hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-
examples-3.3.6.jar \
          pi 4 1000
```

(4 map tasks, 1000 samples per task — good for demonstrating parallelism verbally.)

Grep (pattern search across HDFS input)

```
        hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-
examples-3.3.6.jar \
          grep /wc/input /wc/grepout 'hadoop'
```

---

4. Writing Your Own WordCount (Java) — Likely Practical Task

If the exam asks you to *write* a MapReduce program, this is the canonical skeleton. Know the three classes cold: **Mapper, Reducer, Driver (main)**.

```
        import java.io.IOException;
        import org.apache.hadoop.conf.Configuration;
        import org.apache.hadoop.fs.Path;
        import org.apache.hadoop.io.IntWritable;
        import org.apache.hadoop.io.Text;
        import org.apache.hadoop.mapreduce.Job;
        import org.apache.hadoop.mapreduce.Mapper;
        import org.apache.hadoop.mapreduce.Reducer;
        import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
        import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

        public class WordCount {

          public static class TokenizerMapper
              extends Mapper<Object, Text, Text, IntWritable> {
            private final static IntWritable one = new IntWritable(1);
            private Text word = new Text();

            public void map(Object key, Text value, Context context)
                throws IOException, InterruptedException {
              for (String token : value.toString().split("\\s+")) {
                if (!token.isEmpty()) {
                  word.set(token);
                  context.write(word, one);
                }
              }
            }
          }

          public static class IntSumReducer
              extends Reducer<Text, IntWritable, Text, IntWritable> {
            private IntWritable result = new IntWritable();

            public void reduce(Text key, Iterable<IntWritable> values,
Context context)
                throws IOException, InterruptedException {
              int sum = 0;
              for (IntWritable val : values) sum += val.get();
              result.set(sum);
              context.write(key, result);
            }
          }

          public static void main(String[] args) throws Exception {
            Configuration conf = new Configuration();
```

```
            Job job = Job.getInstance(conf, "word count");
            job.setJarByClass(WordCount.class);
            job.setMapperClass(TokenizerMapper.class);
            job.setCombinerClass(IntSumReducer.class);   // optional but
good to mention
            job.setReducerClass(IntSumReducer.class);
            job.setOutputKeyClass(Text.class);
            job.setOutputValueClass(IntWritable.class);
            FileInputFormat.addInputPath(job, new Path(args[0]));
            FileOutputFormat.setOutputPath(job, new Path(args[1]));
            System.exit(job.waitForCompletion(true) ? 0 : 1);
          }
        }
```

Compiling and packaging it yourself

```
        mkdir -p wordcount_classes
        javac -classpath $(hadoop classpath) -d wordcount_classes
WordCount.java
        jar -cvf wordcount.jar -C wordcount_classes .

        hdfs dfs -rm -r /wc/output          # if it exists
        hadoop jar wordcount.jar WordCount /wc/input /wc/output
        hdfs dfs -cat /wc/output/part-r-00000
```

Concepts to be able to explain verbally: - **Mapper** emits `(key, value)` pairs from each input split. - **Shuffle & Sort** (automatic, between map and reduce) groups all values by key across the cluster. - **Combiner** is a mini-reducer that runs *locally on the mapper's output* before shuffle, cutting network traffic — optional, only safe when the reduce operation is associative/commutative (like sum). - **Reducer** receives `(key, Iterable<values>)` and produces the final output. - **Partitioner** (default: hash of key) decides which reducer each key goes to.

---

5. Hadoop Streaming (Python) — Alternative if Asked for Python

Some labs ask for streaming instead of Java. Know this pattern:

mapper.py

```
        #!/usr/bin/env python3
        import sys
        for line in sys.stdin:
            for word in line.strip().split():
                print(f"{word}\t1")
```

reducer.py

```
        #!/usr/bin/env python3
        import sys
        current_word, current_count = None, 0
        for line in sys.stdin:
            word, count = line.strip().split("\t")
            count = int(count)
            if word == current_word:
                current_count += count
            else:
                if current_word:
                    print(f"{current_word}\t{current_count}")
                current_word, current_count = word, count
        if current_word:
            print(f"{current_word}\t{current_count}")
```

Run it:

```
        chmod +x mapper.py reducer.py
        hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-
3.3.6.jar \
          -input /wc/input \
          -output /wc/output_py \
          -mapper mapper.py \
          -reducer reducer.py
```

---

6. YARN — Monitoring and Control Commands

Command	Purpose
`yarn node -list`	List active NodeManagers
`yarn application -list`	List running/recent applications
`yarn application -status <app_id>`	Detailed status of one application
`yarn application -kill <app_id>`	Kill a running job
`yarn logs -applicationId <app_id>`	Fetch aggregated logs for a finished job
`yarn top`	Live resource usage view (like `top` for YARN)

---

7. Web UIs (know the ports — commonly quizzed)

UI	Default URL	Shows
NameNode Web UI	`http://localhost:9870`	HDFS health, browse filesystem, live/dead DataNodes
ResourceManager Web UI	`http://localhost:8088`	Running/completed YARN applications, cluster resource usage
DataNode Web UI	`http://localhost:9864`	Individual DataNode status
Job History Server	`http://localhost:19888`	Completed MapReduce job details (needs `mr-jobhistory-daemon.sh start historyserver`)

If your exam is on WSL2 and demoed via a browser, these usually work directly from Windows since WSL2 shares `localhost` with the host.

---

8. Troubleshooting — The Fixes Examiners Expect You to Know

A daemon is missing from `jps` 1. Check logs: `cat $HADOOP_HOME/logs/hadoop-*-namenode-*.log` (swap `namenode` for the missing daemon). 2. Common cause: `JAVA_HOME` not set in `hadoop-env.sh` — re-check Step 8 of your install guide. 3. Common cause: SSH not running — `sudo service ssh start`.

“NameNode is in safe mode”

```
hdfs dfsadmin -safemode leave
```

Safe mode is a startup state where HDFS only allows reads, not writes/deletes, while it verifies block reports from DataNodes. It usually clears itself in a few seconds; force it out if stuck.

"Permission denied" writing to HDFS

```
hdfs dfs -chmod -R 777 /target/path
```

Or run as the correct HDFS user.

MapReduce job fails with "Output directory already exists"

```
hdfs dfs -rm -r /wc/output
```

Port already in use when starting daemons Something didn't shut down cleanly last session.

```
stop-yarn.sh
stop-dfs.sh
jps                     # confirm nothing lingering
# if a stray process remains:
kill -9 <pid>
start-dfs.sh
start-yarn.sh
```

DataNode not showing up (classic single-node gotcha) Usually caused by re-running `hdfs namenode -format` after data already existed — the DataNode's `VERSION` file (clusterID) no longer matches the NameNode's. Fix: stop everything, delete the contents of `~/hadoopdata/hdfs/datanode` and `~/hadoopdata/hdfs/namenode`, then re-format and restart. **Only do this if you're okay losing existing HDFS data.**

`hadoop: command not found` `$HADOOP_HOME/bin` and `/sbin` aren't on PATH in this shell — re-source: `source ~/.bashrc`.

---

9. Core Concepts — Rapid-Fire Recap for Viva/Theory Questions

- **Why replication factor 1 on single-node?** Only one DataNode exists; replicating to 3 is physically impossible, so it's set to 1 to avoid perpetual under-replication warnings.
- **NameNode vs DataNode:** NameNode = metadata only (directory tree, block locations). DataNode = actual block storage. Losing NameNode metadata makes the filesystem unreadable even if blocks survive.
- **Secondary NameNode ≠ backup NameNode.** It periodically merges the edit log into `fsimage` to keep restarts fast — it is *not* a failover node.
- **Default block size:** 128 MB in Hadoop 2.x/3.x (was 64 MB in Hadoop 1.x). Large blocks minimize seek overhead relative to transfer time for huge files.
- **Why YARN exists (vs Hadoop 1.x):** Hadoop 1.x's JobTracker did both resource management and job scheduling, becoming a bottleneck and locking the cluster to MapReduce only. YARN split this into ResourceManager (cluster-wide resource arbitration) + ApplicationMaster (per-job scheduling), letting the same cluster run Spark, Tez, etc. alongside MapReduce.
- **Shuffle phase:** the (automatic) transfer and sort of mapper output by key, grouping all values for a given key onto the correct reducer, over the network.
- **Speculative execution:** Hadoop can run duplicate copies of a slow-running task on other nodes and use whichever finishes first — mitigates "straggler" nodes without you having to detect them manually.

- **Fault tolerance in one sentence:** if a DataNode dies, missed heartbeats tell the NameNode, which instructs another DataNode to re-replicate the under-replicated blocks automatically.

---

10. Suggested Exam-Day Command Order (dry run this once before the exam)

```
        # 1. Boot cluster
        sudo service ssh start
        start-dfs.sh
        start-yarn.sh
        jps

        # 2. HDFS sanity check
        hdfs dfs -mkdir -p /exam
        echo "test line one two two three" > t.txt
        hdfs dfs -put t.txt /exam
        hdfs dfs -cat /exam/t.txt

        # 3. Run a built-in job to prove YARN works
        hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-
examples-3.3.6.jar \
          wordcount /exam /exam_out
        hdfs dfs -cat /exam_out/part-r-00000

        # 4. Cluster health check (good to show if asked "prove your cluster
is healthy")
        hdfs dfsadmin -report

        # 5. Clean shutdown at the end
        stop-yarn.sh
        stop-dfs.sh
```

Good luck on the exam — you already have a correctly configured cluster; the rest is muscle memory on these commands.